R.M.K
GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22CS910 - DevOps

Department : Artificial Intelligence & Data Science

Batch / Year : 2022 - 2026 / IV

Created by : Dr. Gladiss Merlin N R, Associate Professor / ADS

Date : 17.05.2025

R.M.K GROUP OF INSTITUTIONS

1. Contents

S. No.	Contents
1	Contents
2	Course Objectives
3	Prerequisites
4	Syllabus
5	Course Outcomes
6	CO-PO Mapping
7	Lecture Plan
8	Activity Based Learning
9	Lecture Notes
10	Assignments
11	Part- A Questions & Answers
12	Part-B Questions
13	Supportive Online Courses
14	Real Time Applications
15	Content beyond the Syllabus
16	Assessment Schedule
17	Prescribed Text books & Reference Books
18	Mini Project Suggestions

R.M.K GROUP OF INSTITUTIONS

Content – Unit II

S.No.	Contents
1	Build production-grade applications
2	MYSQL - mapping Java classes to relational database
3	Introduction to Git version control system
4	Git commands for basic operations (clone, commit, push, pull)
5	Branching strategies
6	Merging strategies
7	Collaboration using Git workflows
8	Revision
9	Revision

2. Course Objectives

The Course will enable learners to:

- Bridge the gap between development and operations for faster, more reliable software releases.
- Automate software delivery with CI/CD pipelines.
- Package and deploy apps efficiently using Docker containers.
- Automate infrastructure with Infrastructure as Code (IaC).
- Monitor and troubleshoot applications in production.

R.M.K GROUP OF INSTITUTIONS

3. Prerequisites

22CS910 - DevOps

22IT910
Rest Application Development Using Spring Boot and JPA

22CS402
Web Development Frameworks

22CS301
Advanced Java Programming

22CS202
Java Programming

22CS101 Problem Solving Using C++
22CS102 Software Development Practices

R.M.K GROUP OF INSTITUTIONS

4. Syllabus

22CS910	DevOps	L	T	P	C
		3	0	0	3

OBJECTIVES:
The Course will enable learners to:

- ❖ Bridge the gap between development and operations for faster, more reliable software releases.
- ❖ Automate software delivery with CI/CD pipelines.
- ❖ Package and deploy apps efficiently using Docker containers.
- ❖ Automate infrastructure with Infrastructure as Code (IaC).
- ❖ Monitor and troubleshoot applications in production.

UNIT I	Introduction to DEVOPS	9

Software Development Methodologies - Operations Methodologies - Systems Methodologies - Development, Release, and Deployment Concepts - Infrastructure Concepts. What is DevOps? - DevOps importance and benefits -DevOps principles and practices - 7 C's of DevOps lifecycle for business agility - DevOps and continuous testing. How to choose right DevOps tools? - Challenges with DevOps implementation.

UNIT II	Version Control with GIT	9

Build production-grade applications – MYSQL - mapping Java classes to relational database - Introduction to Git version control system - Git commands for basic operations (clone, commit, push, pull) - Branching and merging strategies - Collaboration using Git workflows.

UNIT III	Continuous Integration and Delivery (CI/CD)	9

Introduction to CI/CD pipelines - Benefits of CI/CD for faster deployments - Setting up a CI/CD pipeline with Jenkins - Automating builds, tests, and deployments.

UNIT IV	Containerization with Docker	9

Introduction to containerization and its benefits - Understanding Docker concepts: images, containers, registries - Building and managing Docker containers - Docker Compose for multi- container applications - Introduction to container orchestration with Docker Swarm or Kubernetes.

UNIT V	Infrastructure as Code (IaC) and Monitoring	9

Introduction to Infrastructure as Code (IaC) - Benefits of using IaC for repeatable infrastructure provisioning - Learning IaC with Terraform - Setting up infrastructure configurations with Terraform - Introduction to monitoring and logging tools for applications - Alerting and troubleshooting techniques.

TOTAL: 45 PERIODS

R.M.K GROUP OF INSTITUTIONS

4. Syllabus Contd...

OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Understand the core principles and philosophies of DevOps.
CO2: Implement version control systems for code management and collaboration.
CO3: Automate software delivery pipelines using CI/CD tools.
CO4: Utilize containerization technologies for packaging and deploying applications.
CO5: Configure infrastructure as code (IaC) for repeatable deployments.
CO6: Monitor and maintain applications in a production environment.

TEXTBOOKS:

1. Deepak Gaikwad, Viral Thakkar, "DevOps Tools: from Practitioner's Point of View", Wiley, 2019.
2. Jennifer Davis, Ryn Daniels, "Effective DevOps", O'Reilly Media, 2016.

REFERENCES:

1. Gene Kim, Jez Humble, Patrick Debois, "The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations", IT Revolution Press, 2016.
2. Jez Humble, Gene Kim, "Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation", Addison-Wesley, 2010.
3. Yevgeniy Brikman, "Terraform: Up & Running: Writing Infrastructure as Code", O'Reilly Media, 2019.
4. Joseph Muli, "Beginning DevOps with Docker", Packt Publishing, 2018.

R.M.K GROUP OF INSTITUTIONS

5. Course Outcomes

Upon completion of the course, the students will be able to:

CO1: Understand the core principles and philosophies of DevOps.

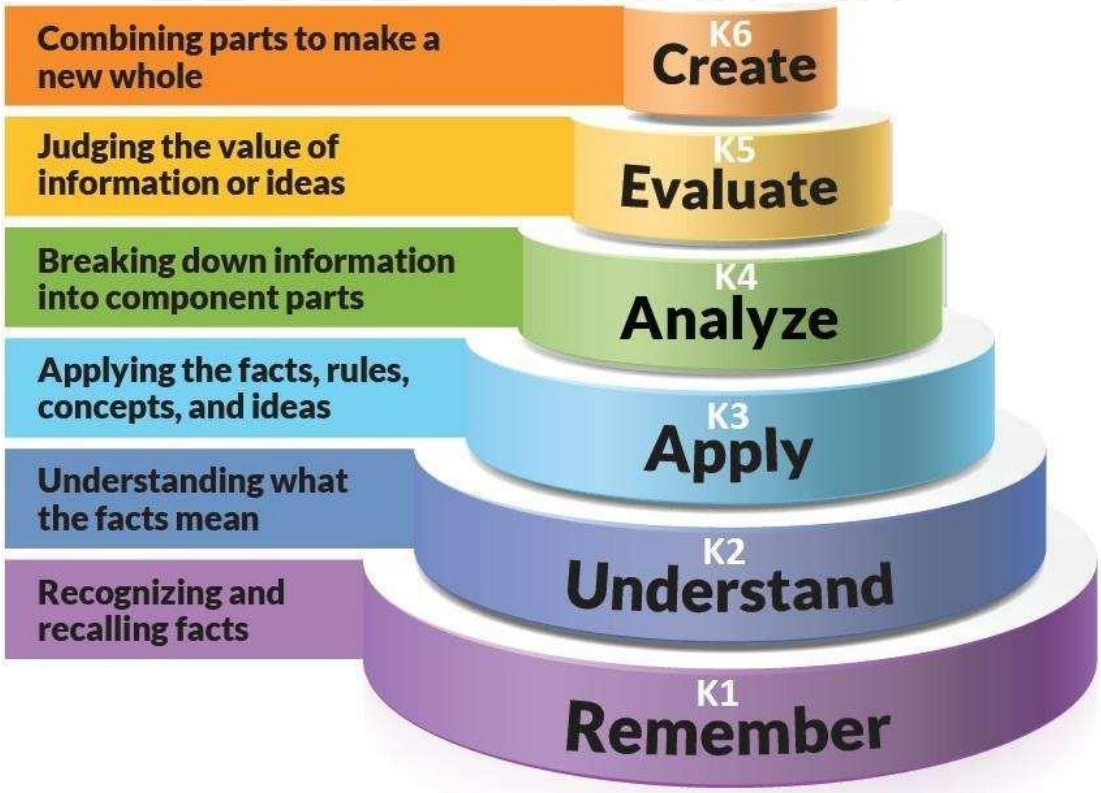
CO2: Implement version control systems for code management and collaboration.

CO3: Automate software delivery pipelines using CI/CD tools.

CO4: Utilize containerization technologies for packaging and deploying applications.

CO5: Configure infrastructure as code (IaC) for repeatable deployments.

CO6: Monitor and maintain applications in a production environment.



R.M.K GROUP OF INSTITUTIONS

6. CO - PO Mapping

	POs and PSOs														
COs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO1	PSO2	PSO3
CO1	3	3	3		2			2	2	1		1			
CO2	3	3	3		2			2	2	1		1			
CO3	3	3	3		2			2	2	1		1			
CO4	3	3	3		2			2	2	1		1			
CO5	3	3	3		2			2	2	1		1			

R.M.K GROUP OF INSTITUTIONS

7. Lecture Plan - Unit I

S. No.	Topic	No. of Periods	Proposed Date	Actual Lecture Date	Pertaining CO	Taxonomy Level	Mode of Delivery
1	Build production-grade applications	1			CO2	K2	Chalk & Talk
2	MYSQL - mapping Java classes to relational database	1			CO2	K3	Chalk & Talk
3	Introduction to Git version control system	1			CO2	K3	Chalk & Talk
4	Git commands for basic operations (clone, commit, push, pull)	1			CO2	K2	Chalk & Talk
5	Branching strategies	1			CO2	K3	Chalk & Talk
6	Merging strategies	1			CO2	K3	Chalk & Talk
7	Collaboration using Git workflows	1			CO2	K2	Chalk & Talk
8	Revision	1			CO2	K3	Chalk & Talk
9	Revision	1			CO2	K2	Chalk & Talk

8. Activity Based Learning

Learning Method	Activity
Learn by Solving Problems	Tutorial Sessions available in RMK Nextgen
Learn by Questioning	Quiz / MCQ Using RMK Nextgen App
Learn by doing Hands-on	Practice available in RMK Nextgen

RMK Nextgen

R.M.K GROUP OF INSTITUTIONS

9. Lecture Notes

Version Control with GIT

Build production-grade applications – MYSQL - mapping Java classes to relational database - Introduction to Git version control system - Git commands for basic operations (clone, commit, push, pull) - Branching and merging strategies - Collaboration using Git workflows.

R.M.K GROUP OF INSTITUTIONS

2.1. Build Production Grade Applications

A production-grade application is one that's reliable, secure, and performs well under various conditions. Modern DevOps practices emphasize continuous deployment, which ensures that the software stays relevant and user-friendly

Building a production-grade application with DevOps support involves integrating robust development practices, automated workflows, and scalable infrastructure. Here's a structured approach to achieving this:

1. Define Requirements and Architecture

Business Goals: Define the purpose, performance, and scalability needs of your application.

Tech Stack: Choose programming languages, frameworks, and databases.

System Architecture: Decide on monolithic or microservices architecture, serverless design, or containerization.

2. Set Up Version Control

Use Git for version control.

Platforms like **GitHub**, **GitLab**, or **Bitbucket** provide collaboration, code reviews, and integration support.

Establish branching strategies such as **Git Flow** or **Trunk-Based Development**.

3. Establish CI/CD Pipelines

Automate the build, test, and deployment processes:

Continuous Integration (CI): Automate testing for every code commit.

Continuous Delivery (CD): Automate deployments to staging or production.

Tools: **Jenkins**, **GitLab CI/CD**, **CircleCI**, **AWS CodePipeline**, or **Azure DevOps**.

R.M.K GROUP OF INSTITUTIONS

4. Infrastructure as Code (IaC)

Use IaC tools to define and provision infrastructure:

Terraform, **AWS CloudFormation**, or **Pulumi**.

Keep infrastructure code in version control for traceability and reproducibility.

5. Containerization and Orchestration

Use **Docker** to package the application and dependencies into containers.

Use **Kubernetes** or **Docker Swarm** for container orchestration to manage scalability and fault tolerance.

6. Automate Testing

Integrate various testing methods:

Unit Tests: Verify individual components.

Integration Tests: Test component interactions.

End-to-End Tests: Validate the entire application flow.

Tools: **JUnit**, **Selenium**, **Cypress**, **Postman**, or **TestNG**.

7. Monitoring and Logging

Set up monitoring tools to track application and infrastructure health:

Metrics: Use **Prometheus** or **Datadog**.

Logs: Use **ELK Stack (Elasticsearch, Logstash, Kibana)** or **Splunk**.

Set up alerts with tools like **PagerDuty** or **Opsgenie**.

8. Security Integration (DevSecOps)

Incorporate security measures early in the lifecycle:

Code Analysis: **SonarQube**, **Snyk**, or **WhiteSource**.

Dependency Scanning: Identify vulnerable libraries.

Penetration Testing: Simulate attacks to find vulnerabilities.

Use tools like **Vault** for managing secrets and credentials.

R.M.K GROUP OF INSTITUTIONS

9. Cloud and Scalability

Choose a cloud provider (**AWS**, **Azure**, **Google Cloud**) for hosting.

Leverage cloud-native tools like **AWS ECS/EKS**, **Azure AKS**, or **Google Kubernetes Engine**.

Implement auto-scaling and load balancing.

10. Blue-Green or Canary Deployments

Ensure smooth rollouts without downtime:

Blue-Green Deployment: Deploy new versions alongside the old and switch traffic gradually.

Canary Deployment: Roll out updates incrementally to a subset of users.

11. Documentation and Knowledge Sharing

Document the architecture, pipelines, and processes.

Use tools like **Confluence** or **Notion** for knowledge sharing.

Provide onboarding documentation for new team members.

12. Feedback Loops and Iteration

Set up mechanisms for gathering feedback (user feedback, performance metrics, incident reports).

Continuously improve processes and workflows.

R.M.K GROUP OF INSTITUTIONS

2.2. MYSQL - Mapping Java classes to Relational Database

Object-relational mapping (ORM) techniques make it easier to work with relational data sources and can bridge your logical business model with your physical storage model. Hibernate can be used to map object-oriented domain models to a traditional relational database.

1. Understanding Object-Relational Mapping (ORM)

ORM simplifies mapping Java objects to relational database tables.

Frameworks like Hibernate or JPA (Java Persistence API) are commonly used.

Each Java class typically corresponds to a database table, with attributes mapped to columns.

2. Setting Up a MySQL Database

Install and configure MySQL.

Create the required schema and tables or use ORM to auto-generate them.

Connect Java to MySQL using the JDBC driver.

3. Define the Java Entity Classes

Annotate Java classes using ORM annotations to map them to tables.

Example with Hibernate/JPA:java

```
 @Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name")
    private String name;

    @Column(name = "email")
    private String email;

    // Getters and setters
}
```

R.M.K GROUP OF INSTITUTIONS

4. Configuring the ORM Framework

Use hibernate.cfg.xml or persistence.xml for configuration:

```
<property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/db_nam
e</property>
<property name="hibernate.connection.username">root</property>
<property name="hibernate.connection.password">password</property>
```

5. Database Operations (CRUD)

Use ORM methods to perform Create, Read, Update, and Delete operations.

Example with Hibernate:

```
Session session = sessionFactory.openSession();
session.beginTransaction();

User user = new User();
user.setName("John Doe");
user.setEmail("john@example.com");

session.save(user);
session.getTransaction().commit();
session.close();
```

6. DevOps Integration

Automation: Automate database setup and migrations with tools like Flyway or Liquibase.

CI/CD: Integrate database schema management into your pipeline.

Testing: Use in-memory databases like H2 for unit testing.

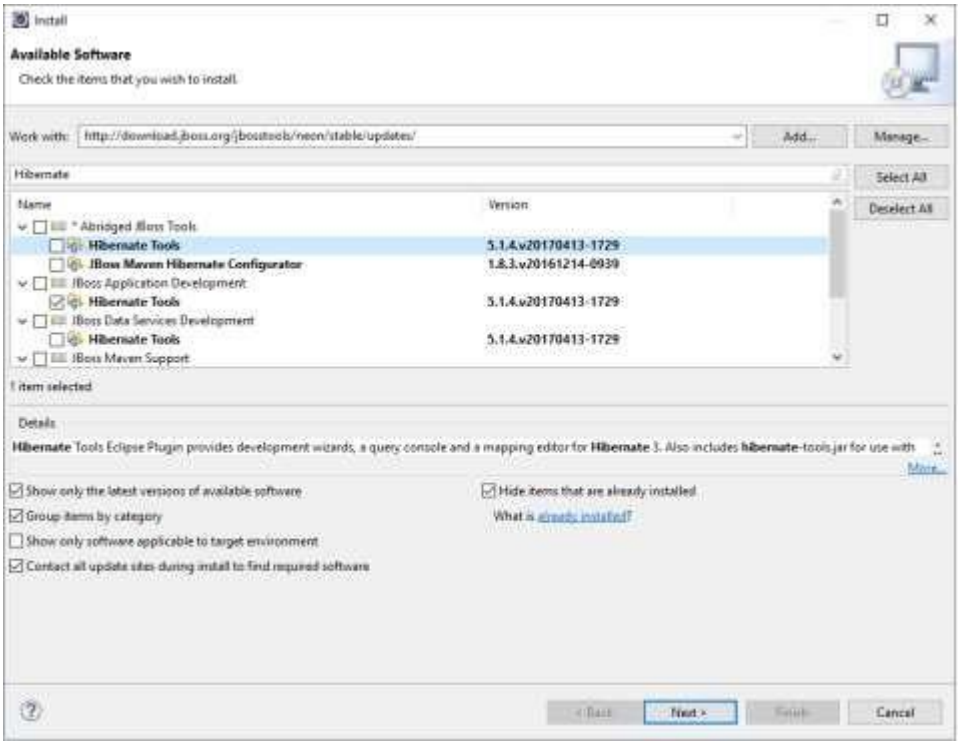
R.M.K GROUP OF INSTITUTIONS

The Practice helps to use the CData JDBC Driver for Azure DevOps to generate an ORM of your Azure DevOps repository with Hibernate.

Install Hibernate

Follow the steps below to install the Hibernate plug-in in Eclipse.

1. In Eclipse, navigate to Help -> Install New Software.
2. Enter "http://download.jboss.org/jbosstools/neon/stable/updates/" in the Work With box.
3. Enter "Hibernate" into the filter box.
4. Select Hibernate Tools.



Start A New Project

Follow the steps below to add the driver JARs in a new project.

1. Create a new project. Select Java Project as your project type and click Next. Enter a project name and click Finish.
2. Right-click the project and click Properties. Click Java Build Path and then open the Libraries tab.
3. Click Add External JARs to add the cdata.jdbc.azuredevops.jar library, located in the lib subfolder of the installation directory.

Add a Hibernate Configuration File

Follow the steps below to configure connection properties to Azure DevOps data.

1. Right-click on the new project and select New -> Hibernate -> Hibernate Configuration File (cfg.xml).
2. Select src as the parent folder and click Next.
3. Input the following values:

Hibernate version:: 5.2

Database dialect: Derby

Driver class: cdata.jdbc.azuredevops.AzureDevOpsDriver

Connection URL: A JDBC URL, starting with jdbc:azuredevops: and followed by a semicolon-separated list of connection properties.

You can connect to your Azure DevOps account by providing the **Organization** and **PersonalAccessToken**.Obtaining a Personal Access Token

A **PersonalAccessToken** is necessary for account authentication.To generate one, log in to your Azure DevOps Organization account and navigate to **Profile -> Personal Access Tokens -> New Token**. The generated token will be displayed.

If you wish to authenticate to Azure DevOps using OAuth refer to the online Help documentation for an authentication guide.

Built-in Connection String Designer

For assistance in constructing the JDBC URL, use the connection string designer built into the Azure DevOps JDBC Driver. Either double-click the JAR file or execute the jar file from the command-line.

```
java -jar cdata.jdbc.azuredevops.jar
```

Fill in the connection properties and copy the connection string to the clipboard.

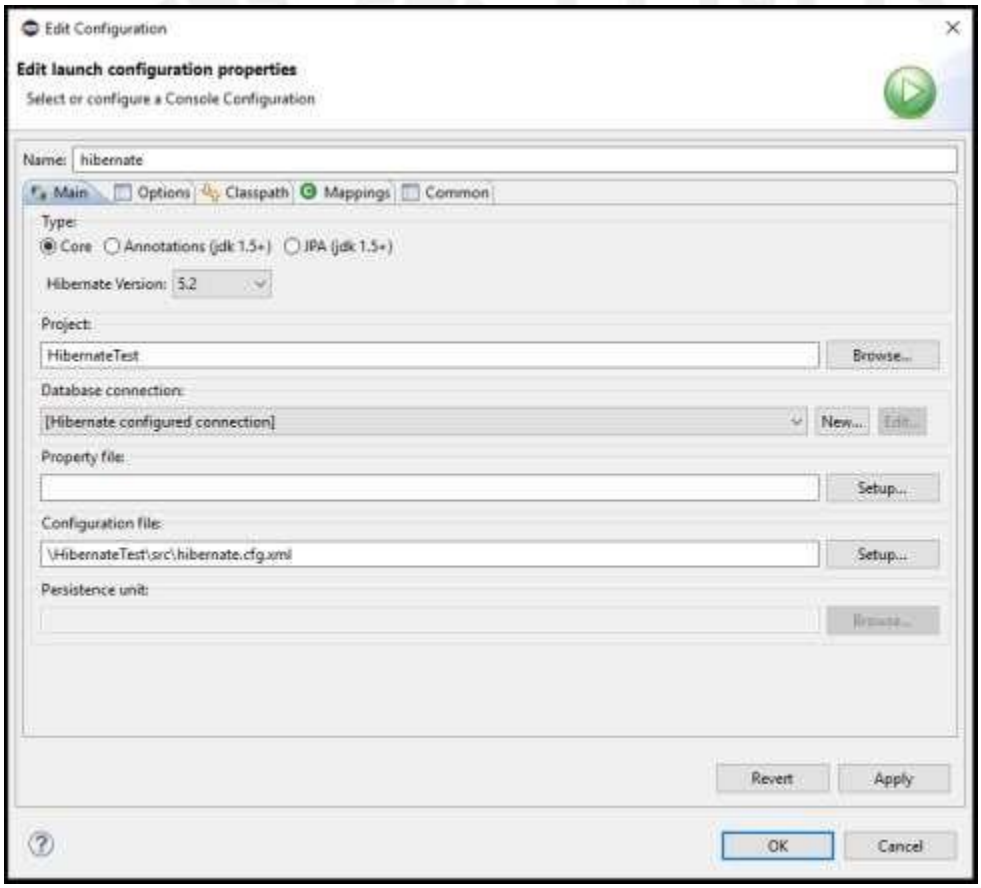
A typical JDBC URL is below:

```
jdbc:azuredevops:AuthScheme=Basic;Organization=MyAzureDevOpsOrganization;ProjectId=MyProjectId;PersonalAccessToken=MyPAT;InitiateOAuth=GETANDREFRESH
```

Connect Hibernate to Azure DevOps Data

Follow the steps below to select the configuration you created in the previous step.

1. Switch to the Hibernate Configurations perspective: Window -> Open Perspective -> Hibernate.
2. Right-click on the Hibernate Configurations panel and click Add Configuration.
3. Set the Hibernate version to 5.2.
4. Click the Browse button and select the project.
5. For the Configuration file field, click Setup -> Use Existing and select the location of the hibernate.cfg.xml file (inside src folder in this demo).
6. In the Classpath tab, if there is nothing under User Entries, click Add External JARS and add the driver jar once more. Click OK once the configuration is done.
7. Expand the Database node of the newly created Hibernate configurations file.

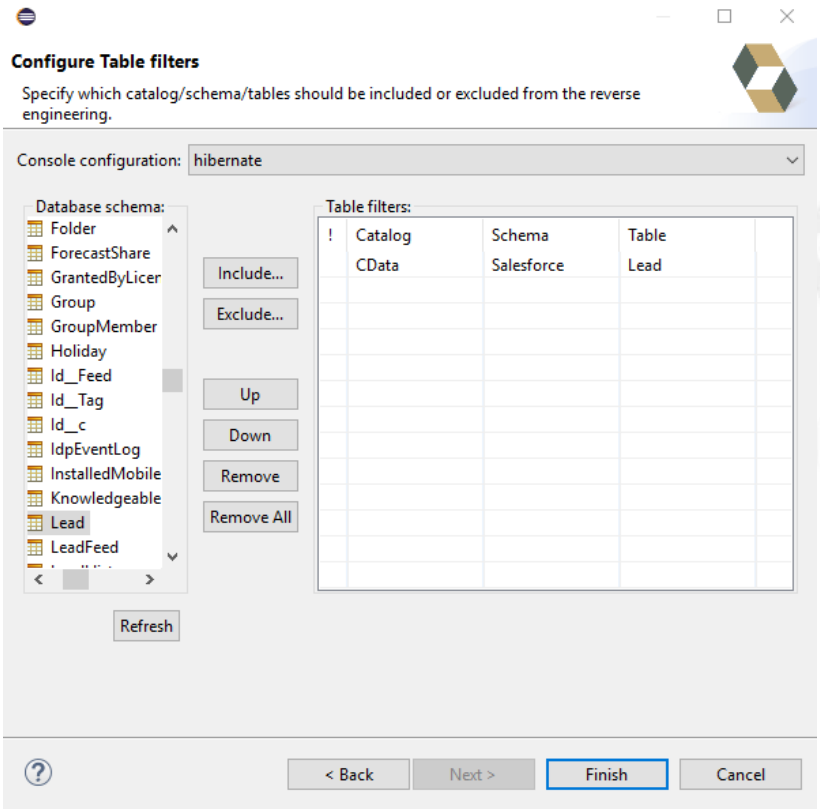


R.M.K GROUP OF INSTITUTIONS

Reverse Engineer Azure DevOps Data

Follow the steps below to generate the reveng.xml configuration file. You will specify the tables you want to access as objects.

1. Switch back to the Package Explorer.
2. Right-click your project, select New -> Hibernate -> Hibernate Reverse Engineering File (reveng.xml). Click Next.
3. Select src as the parent folder and click Next.
4. In the Console configuration drop-down menu, select the Hibernate configuration file you created above and click Refresh.
5. Expand the node and choose the tables you want to reverse engineer. Click Finish when you are done.



Configure Hibernate to Run

Follow the steps below to generate plain old Java objects (POJO) for the Azure DevOps tables.

1. From the menu bar, click Run -> Hibernate Code Generation -> Hibernate Code Generation Configurations.

1. In the Console configuration drop-down menu, select the Hibernate configuration file you created in the previous section. Click Browse by Output directory and select src.
2. Enable the Reverse Engineer from JDBC Connection checkbox. Click the Setup button, click Use Existing, and select the location of the hibernate.reveng.xml file (inside src folder in this demo).
3. In the Exporters tab, check Domain code (.java) and Hibernate XML Mappings (hbm.xml).
4. Click Run.

One or more POJOs are created based on the reverse-engineering setting in the previous step.

Insert Mapping Tags

For each mapping you have generated, you will need to create a mapping tag in hibernate.cfg.xml to point Hibernate to your mapping resource. Open hibernate.cfg.xml and insert the mapping tags as so:

```
cdata.azuredevops.AzureDevOpsDriver
jdbc:azuredevops:AuthScheme=Basic;Organization=MyAzureDevOpsOrganization;ProjectId=MyProjectId;PersonalAccessToken=MyPAT;InitiateOAuth=GETANDREFRESH org.hibernate.dialect.SQLServerDialect
```

Execute SQL

Using the entity you created from the last step, you can now search Azure DevOps data:

```
import java.util.*;
import org.hibernate.Session;
import org.hibernate.cfg.Configuration;
import org.hibernate.query.Query;
public class App {
  public static void main(final String[] args) {
    Session session =  new
Configuration().configure().buildSessionFactory().openSession();
    String SELECT = "FROM Builds B WHERE Reason = :Reason";
    Query q = session.createQuery(SELECT, Builds.class);
    q.setParameter("Reason","Manual");
    List<Builds> resultList = (List<Builds>) q.list();

    for(Builds s: resultList){
      System.out.println(s.getId());
      System.out.println(s.getBuildNumber());
    }
  }
}
```

2.3 Introduction to Git Version Control System

Git is an open source distributed version control system that helps software teams create projects of all sizes with efficiency, speed, and asynchronicity. Git is known for its speed, workflow compatibility, and open source foundation.

Most Git actions only add data to the database, and Git makes it easy to undo changes during the three main states.

Git has three file states: modified, staged, and committed.

A **modified** file has been changed but isn't committed to the database yet.

A **staged** file is set to go into the next commit.

When a file is **committed**, the data has been stored in the database.

With Git, software teams can experiment without worrying that they'll create lasting damage to the source code, because teams can always revert to a previous version if there are any problems.

Version control also known as source control or revision control is software used to track revisions, solve integration conflicts in code, and manage different artifacts involved in software projects (e.g. design, data, images). Version control also enables frictionless communication, change, and reproducibility between developers and other team members.

With version control, it is able to track and merge branches, audit changes, and enable concurrent work to accelerate software delivery. Version control is the foundation and the stepping stone of **DevOps**, software architecture, and release patterns, without which an org cannot start. Using version control results in remarkable team productivity and software quality improvements.

Need to use Git Version Control

Software development teams prefer Git over other version control systems,

like CVS, Mercurial, and Perforce, because Git has the adaptability, speed, and stability. Git version control is the simplest way to empower developers, Also supports to quickly meet customer demand and business goals.

The benefits of Git

The Version Control System is very helpful and beneficial in software development; developing software without using version control is unsafe. It provides backups for uncertainty. Version control systems offer a speedy interface to developers. It also allows software teams to preserve efficiency and agility according to the team scales to include more developers.

Git is fast

Users have a local repository on their machine with a full history, so there's no lag in communicating with a server, which is an experience that users of centralized systems, such as CVS, Subversion, and Perforce, often encounter.

Git is reliable

Git inherently has multiple backups, because each user has a local repository. If there's a crash, a copy could replace the main server. Another benefit of local repositories is that users can continue to commit offline if they're traveling or dealing with network issues.

Git is collaborative

Git makes collaborative development easy with its branching model. People on your team can create a branch, experiment, and merge the code to the main branch if it works well.

Some key benefits of having a version control system are as follows.

Complete change history of the file

Simultaneously working

Branching and merging

Traceability

R.M.K GROUP OF INSTITUTIONS

Types of Version Control System

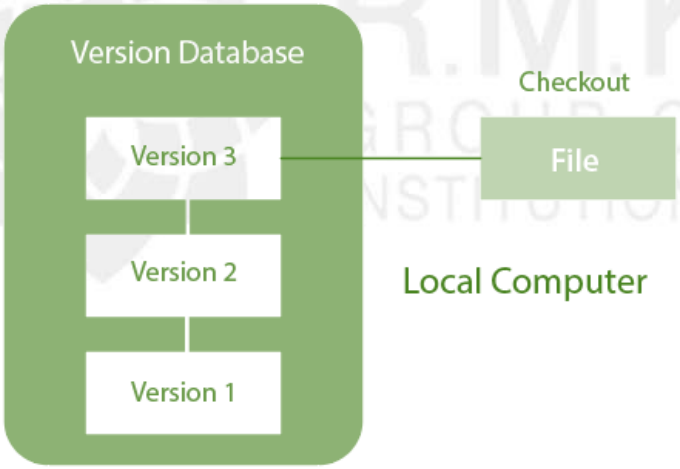
1. Localized Version Control Systems
2. Centralized Version Control Systems
3. Distributed Version Control Systems

Localized Version Control Systems

The localized version control method is a common approach because of its simplicity. But this approach leads to a higher chance of error. In this approach, you may forget which directory you're in and accidentally write to the wrong file or copy over files you don't want to.

To deal with this issue, programmers developed local VCSs that had a simple database. Such databases kept all the changes to files under revision control. A local version control system keeps local copies of the files.

The major drawback of Local VCS is that it has a single point of failure.

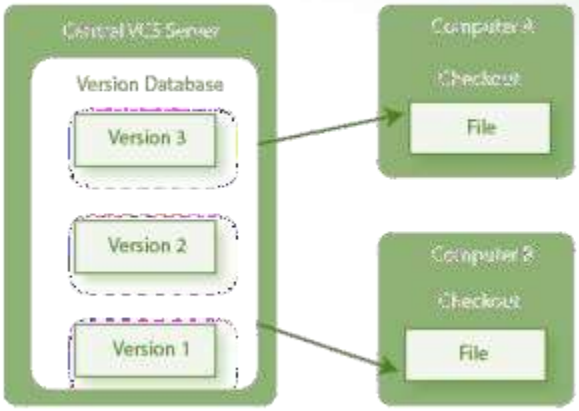


Centralized Version Control System

The developers needed to collaborate with other developers on other systems. The localized version control system failed in this case. To deal with this problem, Centralized Version Control Systems were developed.

These systems have a single server that contains the versioned files, and some clients to check out files from a central place.

R.M.K GROUP OF INSTITUTIONS



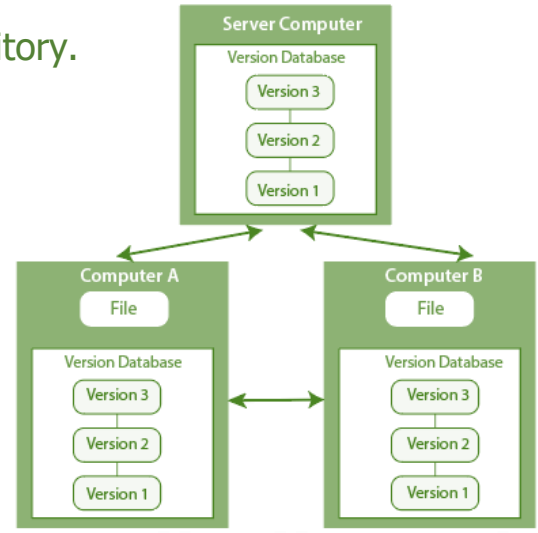
Centralized version control systems have many benefits, especially over local VCSs.

- Everyone on the system has information about the work what others are doing on the project.
- Administrators have control over other developers.
- It is easier to deal with a centralized version control system than a localized version control system.
- A local version control system facilitates with a server software component which stores and manages the different versions of the files.

Distributed Version Control Systems

Centralized Version Control System uses a central server to store all the database and team collaboration. But due to single point failure, which means the failure of the central server, developers do not prefer it. Next, the Distributed Version Control System is developed.

In a Distributed Version Control System (such as Git, Mercurial, Bazaar or Darcs), the user has a local copy of a repository. So, the clients don't just check out the latest snapshot of the files even they can fully mirror the repository. The local repository contains all the files and metadata present in the main repository.



2.4. Git commands for basic operations (clone, commit, push, pull)

Git is a DevOps tool used for source code management. It is a free and open-source version control system used to handle small to very large projects efficiently. Git is used to tracking changes in the source code, enabling multiple developers to work together on non-linear development.

GitHub is a Git repository hosting service that provides a web-based graphical interface (GUI). It helps every team member work together on a project from anywhere, making it easy to collaborate.

Git clone

Creates a copy of an existing Git repository. Cloning is the most common way for developers to obtain a working copy of a central repository.

Git commit

Takes the staged snapshot and commits it to the project history. Combined with git add, this defines the basic workflow for all Git users.

git push

Pushing is the opposite of fetching (with a few caveats). It lets you move a local branch to another repository, which serves as a convenient way to publish contributions. This is like svn commit, but it sends a series of commits instead of a single changeset.

git fetch

Fetching downloads a branch from another repository, along with all of its associated commits and files. But, it doesn't try to integrate anything into your local repository. This gives you a chance to inspect changes before merging them with your project.

git pull

Pulling is the automated version of git fetch. It downloads a branch from a remote repository, then immediately merges it into the current branch. This is the Git equivalent of svn update.

R.M.K GROUP OF INSTITUTIONS

Git clone

In Git, cloning is the act of making a copy of any target repository. The target repository can be remote or local. You can clone your repository from the remote repository to create a local copy on your system. Also, can have sync between the two locations.

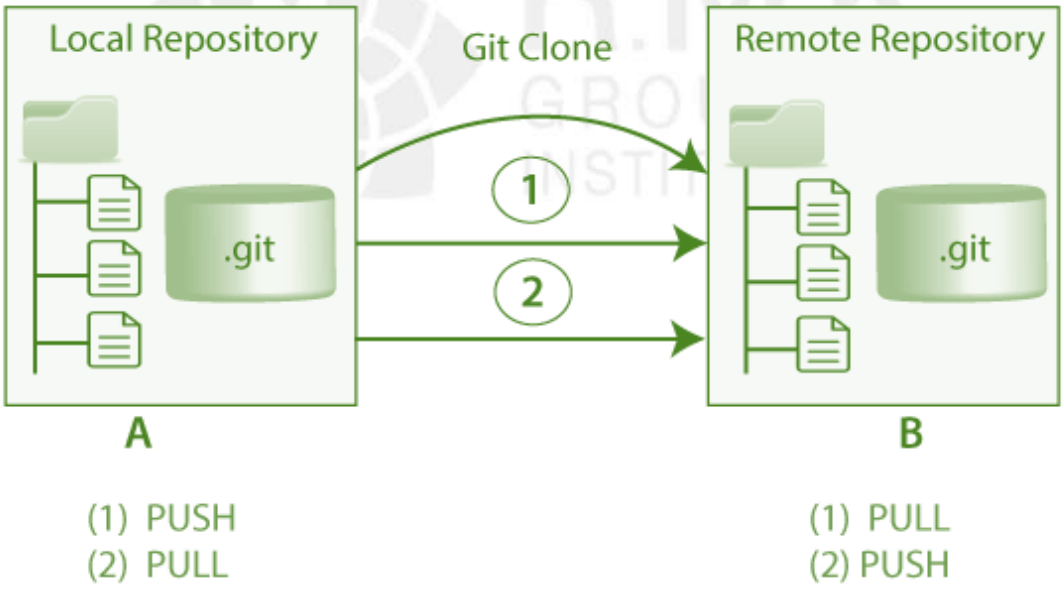
Git Clone Command

The git clone is a command-line utility which is used to make a local copy of a remote repository. It accesses the repository through a remote URL.

Usually, the original repository is located on a remote server, often from a Git service like GitHub, Bitbucket, or GitLab. The remote repository URL is referred to the origin.

Syntax:

$ git clone <repository URL>



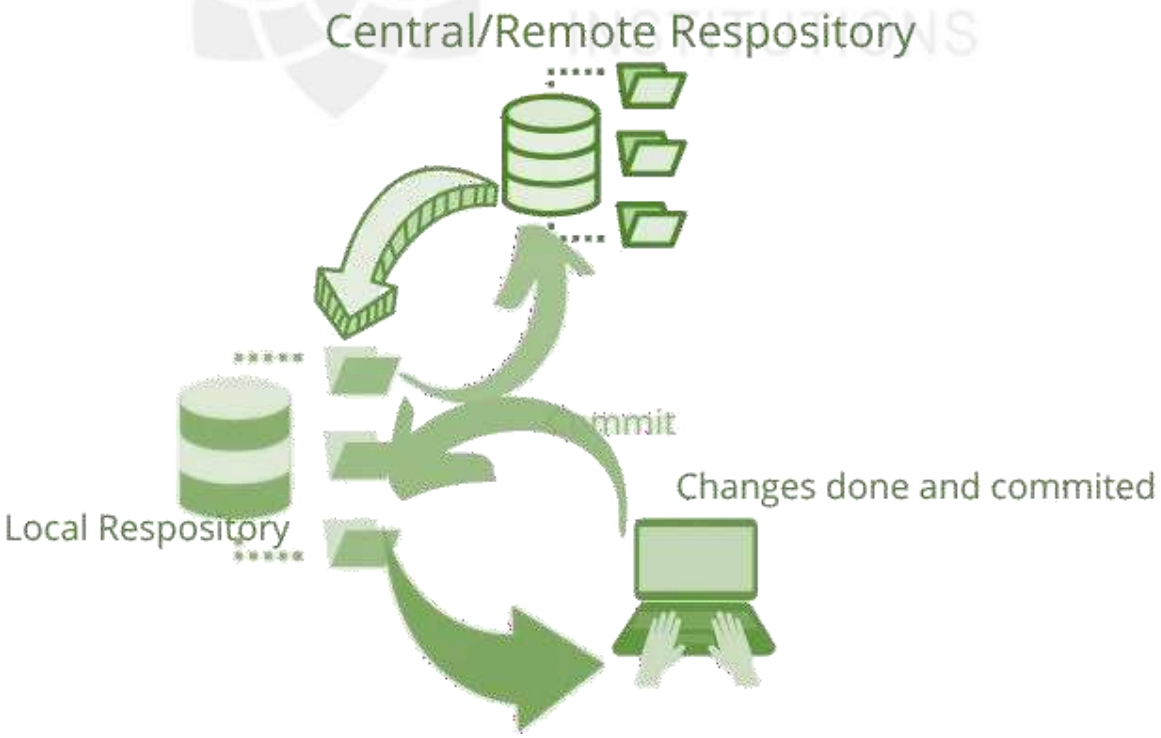
R.M.K GROUP OF INSTITUTIONS

Git commit

Git commit is used to record the changes in the repository. It is the next command after the git add. Every commit contains the index data and the commit message. Every commit forms a parent-child relationship. When we add a file in Git, it will take place in the staging area. A commit command is used to fetch updates from the staging area to the repository.

The staging and committing are co-related to each other. Staging allows us to continue in making changes to the repository, and when we want to share these changes to the version control system, committing allows us to record these changes.

Commits are the snapshots of the project. Every commit is recorded in the master branch of the repository. We can recall the commits or revert it to the older version. Two different commits will never overwrite because each commit has its own commit-id. This commit-id is a cryptographic number created by SHA (Secure Hash Algorithm) algorithm.

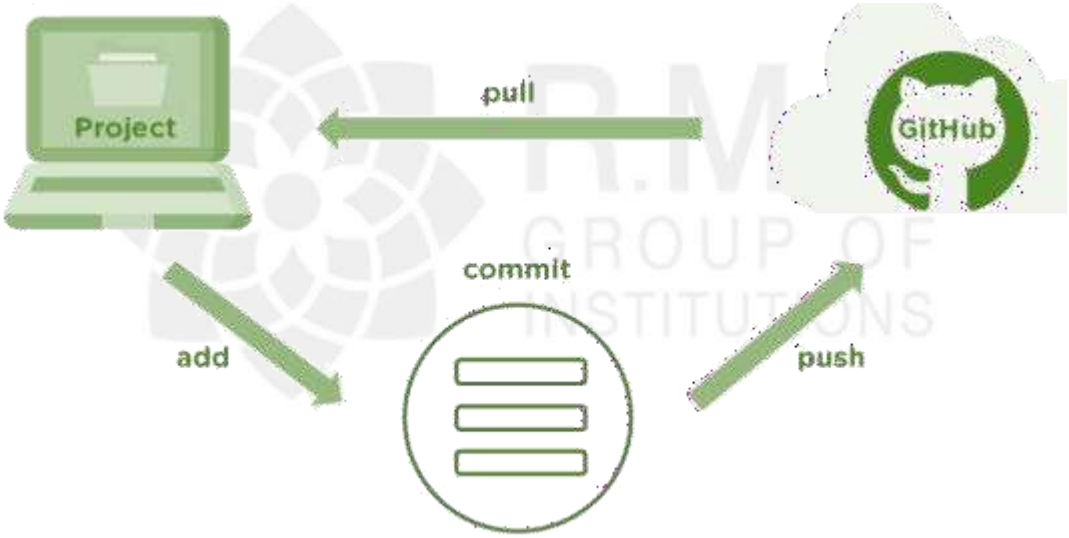


R.M.K GROUP OF INSTITUTIONS

Git Push

The git push command uploads content from a local repository to a remote repository. Pushing refers to the process of moving commits from one repository to another. Pushing is the equivalent of git fetch, except that instead of importing commits to a local branch, it exports commits to an external branch.

The Git push command is used to push the local repository content to a remote repository. After a local repository has been modified, a push is executed to share the modifications with remote team members. Pushing is the way commits are transferred from the local repository to the remote repository.



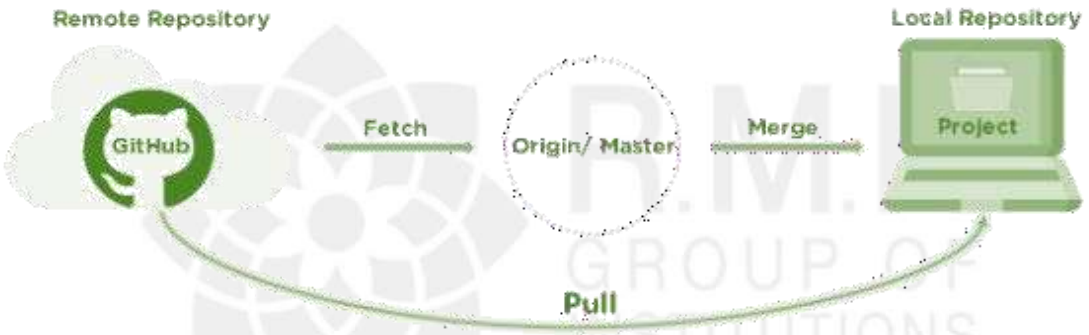
R.M.K GROUP OF INSTITUTIONS

Git Pull

The git pull command is used to retrieve and download content from a remote repository and update the local repository as soon as it has been downloaded. In Git-based collaboration workflows, it is common to merge remote upstream changes into your local repository.

The Git pull command is used to fetch and merge code changes from the remote repository to the local repository. Git pull is a combination of two commands, Git fetch followed by Git merge.

In the first stage, Git fetch is executed that downloads content from the required remote repository. Then, the Git merge command combines multiple sequences of commits into a single branch.



R.M.K GROUP OF INSTITUTIONS

2.5 Branching Strategies

A branching strategy is a strategy that software development teams adopt for writing, merging and deploying code with the help of a version control system like Git.

It lays down a set of rules that aid the developers on how to go about the development process and interact with a shared codebase.

Strategies like these are essential as they help in keeping project repositories organized, error free and avoid the dreaded merge conflicts when multiple developers simultaneously push and pull code from the same repository.

The Branching strategies provides following features:

1. Parallel development
2. Enhanced productivity due to efficient collaboration
3. Organized and structured feature releases
4. Clear path for software development process
5. Bug-free environment without disrupting development workflow

Step By Step Implementation Of Creating A Branch

The following are the steps for creating a branch:

Step 1: Create Branch

Create a branch with the name you want to specify, here we are naming the branch name as "new-feature".

```
git branch new-feature
```

Step 2: Navigate to Branch

Now navigate to the new feature branch from the current branch with the following command:

```
git checkout new-feature
```

(or)

Step 3: Creating And Navigating Branch At A Time

The following one command only helps in creating the branch and navigating to the branch.

```
git checkout -b new-feature
```

Step 4: Check Current Branch

Execute the following command to check the current branch that you're on.

```
git branch
```

Step 5: Delete a Branch

Ensure you are present on the branch you want to delete.

```
git branch -d <branch-to-delete>
```

Common Git Branching Strategies

The following are the common git branching strategies:

Gitflow Workflow

GitFlow enables parallel development, where developers can work separately on feature branches, where a feature branch is created from a master_branch. After completion of changes, the feature branch is merged with the master branch.

The types of branches that can be present in GitFlow are:

Master - Used for product release

Develop - Used for ongoing development

Feature Branching - branches off the develop branch to develop new features.

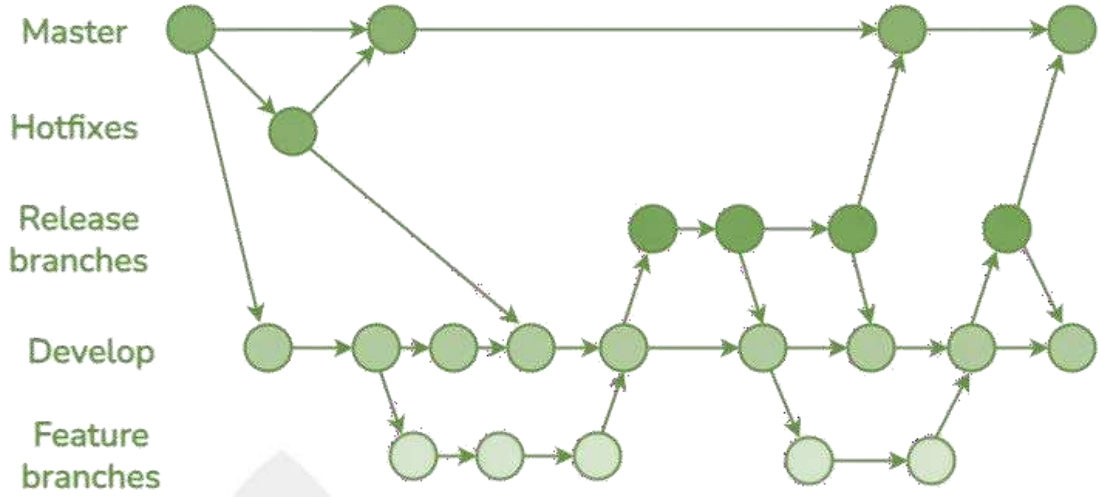
Release - Assist in preparing a new production release and bug fixing, typically branched from the develop branch, and necessitating merges back into both develop and master branches.

Hotfix - Hotfix branches aid in addressing discovered bugs swiftly, allowing developers to continue their work on the develop branch while the issue is

R.M.K GROUP OF INSTITUTIONS

resolved. Unlike release branches, hotfix branches are created from master branch specifically for critical bug resolution in the production release.

The Master and Develop branches are the main branches, and persist throughout the journey of the software. The other branches are essentially supporting branches and are short-lived.



Advantages

1. Facilitates parallel development, ensuring production code stability while developers work on separate branches.
2. Organizes work effectively with separate branches for specific purposes.
3. Ideal for managing multiple versions of production code.
4. GitFlow streamlines the release management process, expediting the rollout of new features and bug fixes.
5. GitFlow offers a well-defined procedure for addressing bugs and deploying hotfixes, facilitating their rapid integration into production environments.

Disadvantages

1. Complexity increases as more branches are added, potentially leading to difficulties in management.

R.M.K GROUP OF INSTITUTIONS

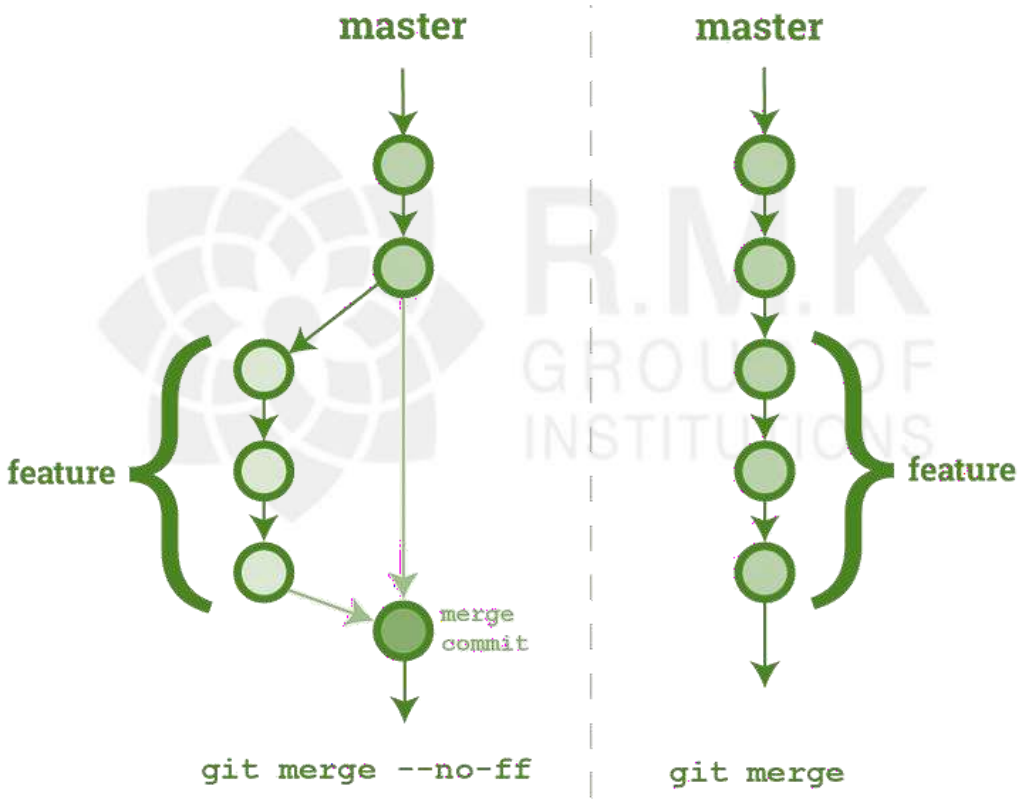
2. Merging changes from development branches to the main branch requires multiple steps, increasing the chance of errors and merge conflicts.
3. Debugging issues becomes challenging due to the extensive commit history.

R.M.K GROUP OF INSTITUTIONS

2.6 Merging Strategies

A **merge** in Git is the process of integrating changes from one branch into another. This is a common practice in collaborative projects where multiple developers work on different features or bug fixes in separate branches. Merging helps combine their efforts into the main branch, usually known as main or master.

Git provides several merge strategies, each suited for different scenarios. The choice of strategy depends on the complexity of changes and the desired outcome. Here are the most commonly used merge strategies:

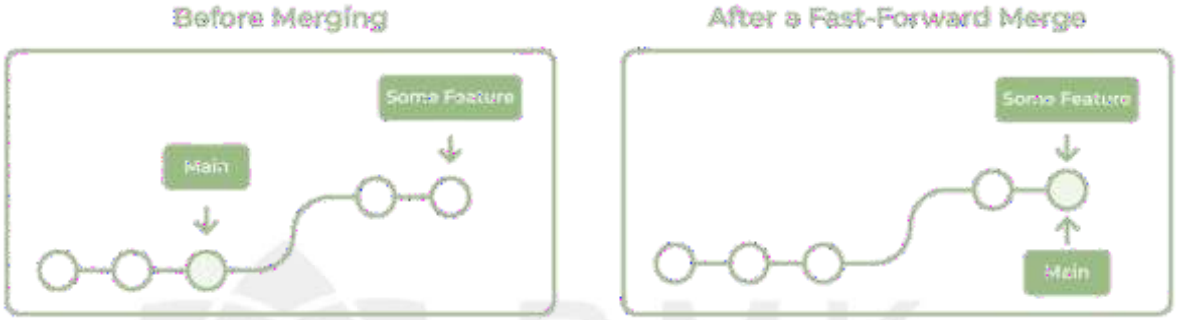


1. Fast Forward Merge:

Fast-forward merge occurs when the target branch has not diverged from the source branch. In this case, Git simply moves the target branch pointer to the latest commit in the source branch. This strategy is simple and keeps the commit history linear.

Command:

```
git checkout main
git merge feature-branch
```

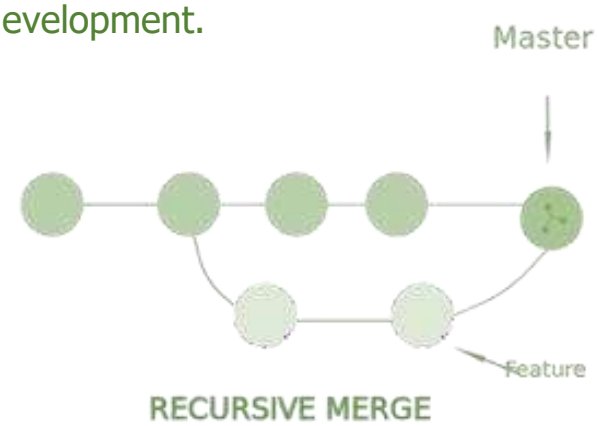


2. Recursive Merge:

Recursive merge is Git's default strategy for non-trivial merges. It handles cases where branches have diverged by creating a new merge commit. This commit records the combined changes from both branches, preserving the history of both lines of development.

Command:

```
$ git merge--no-ff
```



3. Octopus Merge:

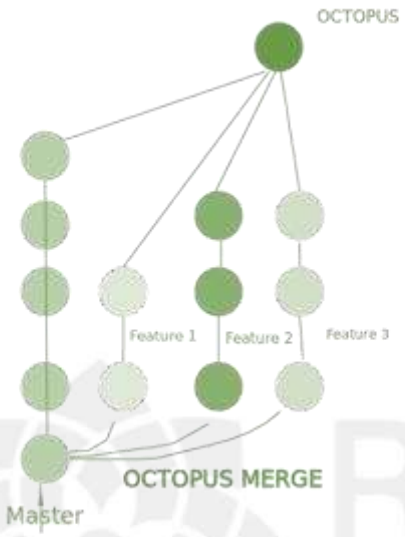
Octopus merge is used for merging more than two branches simultaneously. It's less common and typically used for automated merges involving multiple feature branches.

R.M.K GROUP OF INSTITUTIONS

Octopus Merge strategy resolves cases with more than two heads but refuses to do a complex merge that needs manual resolution. It is primarily meant to be used for bundling topic branch heads together. This is the default merge strategy when pulling or merging more than one branch.

Command:

```
$ git merge -s octopus
```



4. Three-Way Merge

A **three-way merge** involves comparing three commits: the two branch tips and their common ancestor. Git uses this information to create a new merge commit that restore the differences.

Use Case: Useful when dealing with more complex merges involving multiple branches and several changes.

5. Squash and Merge

Squash and merge squashes all the commits from a feature branch into a single commit before merging into the target branch. This strategy simplifies the commit history, making it easier to follow.

R.M.K GROUP OF INSTITUTIONS

Use Case: Ideal for merging feature branches with numerous small commits, resulting in a cleaner main branch history.

```
git checkout main
git merge --squash feature-branch
git commit -m "Merged feature-branch with squash"
```

6. Rebase and Merge

Rebase and merge rewrites the commit history of the feature branch, placing it on top of the main branch before merging. This results in a linear commit history without merge commits.

Use Case: Suitable for maintaining a clean and linear project history, especially in projects with strict commit guidelines.

```
git checkout feature-branch
git rebase main
git checkout main
git merge feature-branch
```

R.M.K GROUP OF INSTITUTIONS

2.7 Collaboration using Git workflows.

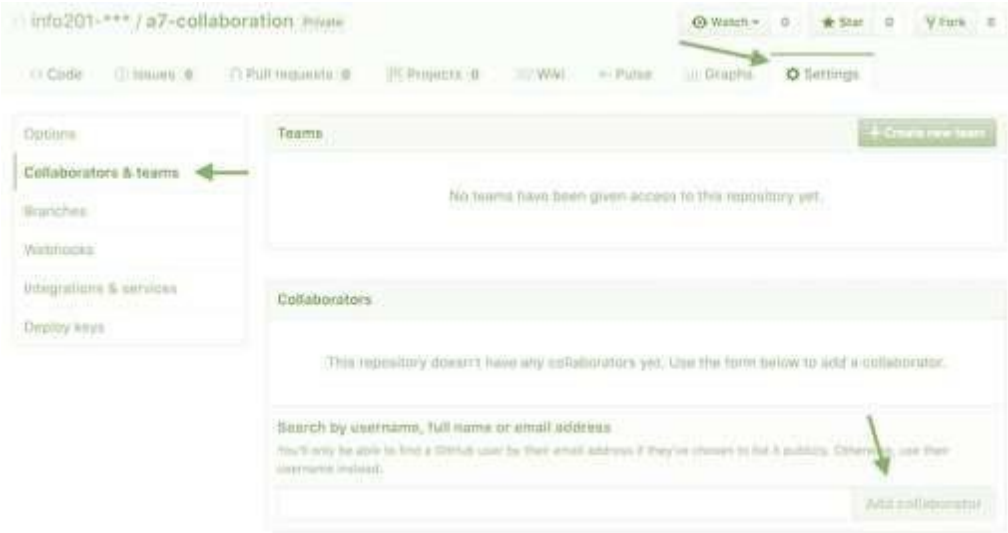
Being able to merge between branches allows you to work **collaboratively**, with multiple people making changes to the same repo and sharing those changes through GitHub. There are a variety of approaches (or **workflows**) that can be used to facilitate collaboration and make sure that people are effectively able to share code.

2.7.1 Centralized Workflow

In order to understand the Feature Branch Workflow, it's important to first understand how to collaborate on a centralized repository. The Feature Branch Workflow uses a **centralized repository** stored on GitHub—that is, every single member of the team will push and pull to a single GitHub repo. However, since each repository needs to be created under a particular account, this means that a ***single member*** of the team will need to create the repo (such as by accepting a GitHub Classroom assignment, or by clicking the "New" button on their "Repositories" tab on the GitHub web portal).

In order to make sure everyone is able to push to the repository, whoever creates the repo will need to **add the other team members as collaborators**.

You can do this under the Settings tab:



Once you've added everyone to the GitHub repository, **each team member** will need to **clone** the repository to their local machines to work on the code individually. Collaborators can then push any changes they make to the central repository, and pull and changes made by others. Because multiple members will be contributing to the same repositiory, it's important to ensure that you are working on the most up-to-date version of the code. This means that you will regularly have to **pull in changes** from GitHub that your team members may have committed. As a result, we suggest that you have a workflow that looks like this:

```
# Begin your work session by pulling in changes from GitHub
git pull origin master
# If necessary, resolve any merge conflicts and commit them
git add .
git commit -m "Merge in changes from GitHub"
# Do your work, then add, commit and push
git add .
git commit -m "Make progress on feature X"
git push origin master
```

Note, if someone pushes a commit to GitHub before you push your changes, you'll need to integrate those into your code (and test them!) before pushing up to GitHub. While working on a single master branch in this fashion is possible, you'll encounter fewer conflicts if you use a dedicated feature branch for each developer or feature you're working on

R.M.K GROUP OF INSTITUTIONS

2.7.2 Feature Branch Workflow

The Feature Branch Workflow is a natural extension of the Centralized Workflow that enhances the model by defining specific branches for different pieces of development (still with one centralized repository). The core idea behind the Feature Branch Workflow is that all development should take place on a dedicated **feature branch**, rather than on the master branch. This allows for different people to work on different branches without disturbing the main codebase. For example, you might have one branch visualization that focuses on adding a complex visualization, or another experimental-analysis that tries a bold new approach to processing the data. Each branch is based on a feature (capability or part) of the project, not a particular person: a single developer could be working on multiple feature branches.

The idea is that the master branch always contains "production-level" code: valid, completely working code that you could deploy or publish (read: give to your boss or teacher) at a whim. All feature branches branch off of master, and are allowed to contain temporary or even broken code (since they are still in development). This way there is always a "working" (if incomplete) copy of the code (master), and development can be kept isolated and considered independent of the whole. This is similar to the example with the experiment branch above.

R.M.K GROUP OF INSTITUTIONS

The workflow thus works like this:

1. Ada decides to add a new feature or part to the code. She creates a new feature branch off of master:

```
git checkout master
git checkout -b adas-feature
```

2. Ada does some work on this feature

```
# work is done outside of terminal
git add .
git commit -m "Add progress on feature"
```

3. Ada takes a break, pushing her changes to GitHub

```
git push -u origin adas-feature
```

4. After talking to Ada, Bebe decides to help finish up the feature. She checks out the branch and makes some changes, then pushes them back to GitHub

```
# fetch will "download" commits from GitHub, without merging them
git fetch origin
git checkout adas-feature
# work is on adas-feature done outside of terminal
git add .
git commit -m "Add more progress on feature"
git push origin adas-feature
```

5. Ada downloads Bebe's changes

```
git pull origin adas-feature
```

6. Ada decides the feature is finished, and merges it back into master. But first, she makes sure she has the latest version of the master code to integrate her changes with

```
git checkout master # switch to master
git pull origin master # download any changes
```

R.M.K GROUP OF INSTITUTIONS

```
git merge adas-feature # merge the feature into the master branch
# fix any merge conflicts!!
git push origin master # upload the updated code to master
```

7. And now that the feature has been successfully added to the project, Ada can delete the feature branch (using git branch -d branch_name).

2.7.3 Forking Workflow

The Forking Workflow takes a fundamentally different approach to collaboration than the Centralized and Feature Branch workflows. Rather than having a single remote, each developer will have their own repository on GitHub that is forked from the original repository. As discussed in the introductory GitHub Chapter, a developer can create their own remote repository from an existing project by forking it on GitHub. This allows the individual to make changes (and contribute to) the project. However, we have not yet discussed how those changes can be integrated into the original code base. GitHub offers a feature called pull requests by which you can merge two remote branches (that is: merge two branches that are on GitHub). A pull request is a request for the changes from one branch to be pulled (merged) into another.

Pull Requests

Pull requests are primarily used to let teams of developers collaborate—one developer can send a request "hey, can you integrate my changes?" to another. The second developer can perform a code review: reviewing the proposed changes and making comments or asking for corrections to anything they find problematic. Once the changes are improved, the pull request can be accepted and the changes merged into the target branch. This process is how programmers collaborate on open-source software

(such as R libraries like dplyr): a developer can fork an existing professional project, make changes to that fork, and then send a pull request back to the original developer asking them to merge in changes

In order to issue a pull request, both branches you wish to merge will need to be pushed to GitHub (whether they are in the same repo or in forks). To issue the pull request, navigate to your repository on GitHub's web portal and choose the New Pull Request button.

In the next page, you will need to specify which branches you wish to merge. The base branch is the one you want to merge into (often master), and the head branch (labeled "compare") is the branch with the new changes you want to merge (often a feature branch; see below).

Add a title and description for your pull request. These should follow the format for git commit messages. Finally, click the Create pull request button to finish creating the pull request.

R.M.K GROUP OF INSTITUTIONS

10. Assignment Questions

S.No.	Topic	K - Level	COs
1	How does Git Supports DevOps in Software Development Process.	K3	CO2
2	How mapping Java classes to relational database is done with the support of Devops	K3	CO2

R.M.K GROUP OF INSTITUTIONS

11. Part A
Question & Answer

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Part A

1. What is Production Grade Application? (CO2, K1)

A production-grade application is one that's reliable, secure, and performs well under various conditions. Modern DevOps practices emphasize continuous deployment, which ensures that the software stays relevant and user-friendly

2. What is support of Devops in Production Grade Application? (CO2, K2)

Building a production-grade application with DevOps support involves integrating robust development practices, automated workflows, and scalable infrastructure.

3. List out the steps in building Production Grade Application with Devops (CO2, K1)

1. Define Requirements and Architecture
2. Set Up Version Control
3. Establish CI/CD Pipelines
4. Infrastructure as Code (IaC)
5. Containerization and Orchestration
6. Automate Testing
7. Monitoring and Logging
8. Security Integration (DevSecOps)
9. Cloud and Scalability
10. Blue-Green or Canary Deployments
11. Documentation and Knowledge Sharing
12. Feedback Loops and Iteration

4. Define ORM. (CO2, K1)

Object-relational mapping (ORM) techniques make it easier to work with relational data sources and can bridge your logical business model with your physical storage model. Hibernate can be used to map object-oriented domain models to a traditional relational database.

5. List out the steps in Object Relational Mapping (CO2, K1)

1. Understanding Object-Relational Mapping (ORM)
2. Setting Up a MySQL Database
3. Define the Java Entity Classes
4. Configuring the ORM Framework
5. Database Operations (CRUD)
6. DevOps Integration

R.M.K GROUP OF INSTITUTIONS

Part A

6. What is Git? (CO2, K1)

Git is an open source distributed version control system that helps software teams create projects of all sizes with efficiency, speed, and a synchronicity. Git is known for its speed, workflow compatibility, and open source foundation.

7. What is the use of Git? (CO2, K1)

Most Git actions only add data to the database, and Git makes it easy to undo changes during the three main states.

8. How many file states Git has? List out. (CO2, K1)

Git has **three** file states: **modified, staged, and committed.**

A **modified** file has been changed but isn't committed to the database yet.

A **staged** file is set to go into the next commit.

When a file is **committed**, the data has been stored in the database.

9. Explain about Git Version Control functionality (CO2, K2)

Version control also known as source control or revision control is software used to track revisions, solve integration conflicts in code, and manage different artifacts involved in software projects (e.g. design, data, images). Version control also enables frictionless communication, change, and reproducibility between developers and other team members.

10. State the Need to use Git Version Control in Software Development (CO2, K2)

Software development teams prefer Git over other version control systems, like CVS,Mercurial, and Perforce, because Git has the adaptability, speed, and stability. Git version control is the simplest way to empower developers, Also supports to quickly meet customer demand and business goals.

11. Enumerate the benefits of Git in Devops. (CO2, K2)

The Version Control System is very helpful and beneficial in software development; developing software without using version control is unsafe. It provides backups for uncertainty. Version control systems offer a speedy interface to developers. It also allows software teams to preserve efficiency and agility according to the team scales to include more developers.

Part A

12. List out the types of Version Control System in Git (CO2, K1)

Types of Version Control System

1. Localized Version Control Systems
2. Centralized Version Control Systems
3. Distributed Version Control Systems

13. Give Brief note on the types of Version Control System (CO2, K1)

- **Local Version Control System**: A simple form of VCS that uses a database to keep track of changes to files
- **Centralized Version Control System**: A system with a single repository where all users must commit to reflect their changes
- **Distributed Version Control System**: A system where each developer has a full copy of the repository on their computer, allowing them to work offline

14. What are some examples of Version Control System ? (CO2, K2)

Some examples of **Version Control System** include:

- **Git**: A free, open-source, distributed VCS used for software development and managing any type of file
- **Apache Subversion**: An open-source VCS that includes features like inventory management, security management, and history tracking
- **GitHub**: A web-based platform for hosting Git repositories
- **GitLab**: A web-based platform similar to GitHub that's often used for private and enterprise projects

15. What is a commit in Git? (CO2, K1)

A commit is a snapshot of the project's changes at a specific point in time, stored in the repository.

16. Explain the difference between centralized and distributed version control systems. (CO2, K1)

Centralized VCS: Has a single central server that stores all the project's code and history. Clients checkout the code from the server, make changes, and commit them back to the central server. Examples: Subversion (SVN), CVS.

Distributed VCS: Each developer has a complete copy of the repository, including the entire history. Changes are made locally and then shared with others. Examples: Git, Mercurial.

R.M.K GROUP OF INSTITUTIONS

Part A

17. What is a repository in a Version Control System? (CO2, K1)

A repository is a central location where all the files and history of a project are stored. It's like a database for code, tracking every change and allowing you to access past versions.

18. What is a commit in a VCS? (CO2, K1)

A commit is a snapshot of the project's code at a specific point in time. It includes a message describing the changes made and the author of the commit.

19. What is the difference between a commit and a push in Git? (CO2, K1)

- Commit: Saves changes locally to your own repository.
- Push: Sends your local commits to the remote repository (usually on a server).

20. What is a branch in a VCS? (CO2, K1)

A branch is a separate line of development that allows developers to work on new features or bug fixes without affecting the main codebase. It's like creating a copy of the project where you can experiment and make changes.

21. What is merging in a VCS? (CO2, K1)

Merging is the process of combining changes from one branch into another. This is usually done when a feature is complete and ready to be integrated with the main codebase.

22. What is a conflict in a VCS? (CO2, K1)

A conflict occurs when two or more developers make changes to the same lines of code in a file. The VCS cannot automatically merge these changes, so it flags them as conflicts and requires manual intervention.

23. What is a pull request in Git?

A pull request is a mechanism in Git to propose changes from a branch to the main codebase. It allows other developers to review the changes before they are merged into the main branch.

24. What are some common Git commands?

- `git init`: Initializes a new Git repository.
- `git clone`: Creates a local copy of a remote repository.
- `git add`: Stages changes for commit.
- `git commit`: Saves changes locally to the repository.
- `git push`: Sends local commits to the remote repository.
- `git pull`: Downloads changes from the remote repository and merges them into the local branch.
- `git branch`: Creates, lists, or deletes branches.
- `git checkout`: Switches to a different branch.
- `git merge`: Merges changes from one branch into another.
- `git status`: Shows the current state of the repository.
- `git log`: Displays the commit history.

R.M.K GROUP OF INSTITUTIONS

Part A

25. What are some best practices for using Git? (CO2, K3)

- **Commit Often and Early:** Make small, frequent commits with clear commit messages.
- **Use Descriptive Commit Messages:** Write clear and concise messages that explain the purpose of the changes.
- **Keep Branches Small and Focused**: Create separate branches for specific features or bug fixes.
- **Review and Test Changes Carefully:** Make sure to review and test changes before merging them into the main branch.
- **Resolve Conflicts Promptly:** Address conflicts quickly to avoid delays in the development process.
- **Use Git Hooks:** Automate certain tasks (like code linting) to ensure code quality.

26. What is a Git workflow ? (CO2, K2)

A Git workflow is a set of guidelines and practices that define how a team uses Git to manage their development process. Different workflows (like Gitflow or GitHub Flow) offer different approaches to branching, merging, and collaboration.

27. What are some of the advantages of using Git over other version control systems ? (CO2, K3)

- Git's advantages include:
- Distributed nature: Allows each developer to have a complete copy of the repository, enabling offline work and faster development.
- Branching and merging: Powerful branching and merging capabilities make it easy to work on separate features and integrate them smoothly.
- Open source and free: Available for free and has a large community of developers contributing to its development.
- Excellent tooling: Git has a wide range of tools and integrations, making it versatile for various development workflows.

28. What are some of the features that make Git a popular choice for version control? (CO2, K3)

- Git's popularity stems from features like:
- Speed and efficiency: Git is designed to be fast and efficient, even with large repositories.
- Branching and merging: Powerful branching and merging capabilities make it easy to work on multiple features concurrently.
- Open source and free: Available for free and has a vast community of developers.
- Extensive tooling: A wide range of tools and integrations are available for Git.
- Versatility: Git can be used for various projects, from small personal projects to large enterprise-scale development.

Part A

29. What are some of the future trends in version control systems ? (CO2, K3)

- Future trends include:
- **Integration with cloud platforms:** Increasing integration with cloud services like AWS and Azure.
- **Improved collaboration features:** Enhanced tools for real-time collaboration and code review.
- **AI-powered features:** Use of AI to automate tasks, suggest changes, and improve code quality.
- **Focus on security**: Enhanced security features to protect sensitive code and data.
- **Support for new languages and frameworks:** Increased support for emerging programming languages and frameworks.

30. What are the benefits of using GitHub Actions for CI/CD? (CO2, K3)

- **Increased automation:** Automating repetitive tasks reduces manual errors and saves time.
- **Improved code quality:** Automated testing and code analysis help improve the quality of your code.
- **Faster delivery:** Automated deployment pipelines shorten release cycles and get features to users more quickly.
- **Enhanced collaboration:** GitHub Actions facilitates collaboration between developers, testers, and operations teams.

R.M.K GROUP OF INSTITUTIONS

12. PART – B Questions

1. Explain in detail about Building Production Grade Applications with the support of Devops. **(CO2, K2)**
2. Give a brief note on MYSQL Database. **CO2, K3)**
3. Explain with the necessery steps to do Mapping Java classes to Relational Database **CO2, K3)**
4. Discus about Git Version Control System **CO2, K2)**
5. List out an explain Git commands for basic operations like clone, commit, push, pull. **CO2, K2)**
6. Write in detail about Branching and Merging Strategies **CO2, K2)**
7. Explain about Collaboration using Git workflows in Software Development. **CO2, K3)**

Note : All the above questions requires Diagrams to Illustrate the Concepts, Wherever Necessary.

R.M.K GROUP OF INSTITUTIONS

13. Supportive online courses

Online Courses

1. https://www.coursera.org/learn/introduction-to-microsoft-github-copilot
2. https://www.coursera.org/learn/introduction-git-github
3. ttps://www.coursera.org/learn/introduction-to-microsoft-github-copilot
4. https://www.udemy.com/course/github-git/

External Links for Additional Resources

1. https://www.w3schools.com/git/
2. https://www.javatpoint.com/git
3. https://www.geeksforgeeks.org/version-control-systems/

R.M.K GROUP OF INSTITUTIONS

14. Real Time Applications

Here are some DevOps tools that use Version Control and other technologies to help with real-time applications:

1. **Git :** A version control system that helps development teams track and manage changes to their codebase.
2. **GitHub Actions :** Allows developers to automate repetitive tasks, such as testing, building, and deploying software.
3. **Docker :** A lightweight tool that helps DevOps teams design, ship, and operate container-based distributed applications
4. **Ansible :** A tool for orchestration, automation, configuration, and managing IT infrastructures
5. **Erraform :** An IaC tool that helps you deploy and manage your infrastructure using a declarative language
6. **Chef :** Provides a simple language for describing infrastructure, allowing you to specify how each component should be configured.
7. **Gradle :** A build automation tool that automates the process of creating applications
8. **Nagios :** An open-source monitoring system that helps organizations monitor and manage their IT infrastructure.
9. **Splunk :** A real-time data monitoring tool that provides insights across all the stages of the delivery lifecycle.

R.M.K GROUP OF INSTITUTIONS

15. Content Beyond Syllabus

DevOps Application and Use Cases

Automation Testing :

Automation Testing is a crucial aspect of DevOps, enabling teams to automatically run a suite of tests against the application with every code change. Automated tests include unit tests, integration tests, and end-to-end tests that help identify and fix bugs early, reducing the chances of introducing defects into the production environment. By automating the testing process, developers can iterate faster, ensure better code quality, and maintain a high level of confidence in the application's performance and functionality.

Infrastructure as Code (IaC):

Infrastructure as Code (IaC) is a DevOps practice that involves defining and managing infrastructure elements, such as servers, networks, and databases, through machine-readable configuration files. These files are version-controlled and treated like code, allowing teams to automate the provisioning and configuration of infrastructure. IaC provides numerous benefits, including consistency in environments, easier replication, faster recovery from failures, and improved collaboration between development and operations teams.

Microservices Deployment:

Microservices Deployment is a software architecture approach where applications are built as a collection of small, loosely coupled services. Each service represents a specific business capability and can be developed, deployed, and scaled independently. DevOps plays a critical role in managing the deployment of microservices, ensuring seamless integration and continuous delivery of new features.

R.M.K GROUP OF INSTITUTIONS

15. Content Beyond Syllabus

This approach fosters scalability, flexibility, and faster development cycles, making it ideal for complex and rapidly evolving applications.

Containerization with Docker:

Containerization with Docker has revolutionized the way applications are deployed and managed. Docker allows developers to package applications and their dependencies into lightweight, portable containers, ensuring consistency across different environments. DevOps teams leverage Docker to streamline the deployment process, simplify versioning, and reduce the risk of environment-related issues. Containerization also enables efficient resource utilization, making it easier to scale applications as needed.

Monitoring and Logging:

Monitoring and Logging are crucial components of any DevOps practice. These tools provide real-time visibility into the application's performance, resource utilization, and potential issues. By proactively monitoring application metrics and logs, DevOps teams can identify bottlenecks, detect anomalies, and respond to incidents promptly. Continuous monitoring helps maintain high availability, optimal performance, and a positive user experience. These practices and applications can be easily mastered by enrolling in any practical-based DevOps certification training course.

Collaborative Development:

DevOps promotes a culture of collaboration and cross-functional teamwork. Collaboration tools, such as chat platforms and issue trackers, facilitate effective communication between developers

operations, and other stakeholders. By breaking down silos and encouraging knowledge sharing, DevOps empowers teams to align their efforts, tackle challenges together, and deliver value to customers more efficiently.

Security and Compliance:

Security is a top priority in software development, and DevOps actively incorporates security measures throughout the development process. Security professionals work closely with development teams to identify and address vulnerabilities early in the software development lifecycle. This ensures that security is not an afterthought but an integral part of the application's design and implementation. Additionally, DevOps helps organizations maintain compliance with relevant regulations and industry standards.

Cloud Infrastructure Management:

DevOps thrives in cloud environments, where it leverages cloud services to manage and scale infrastructure resources dynamically. Cloud providers offer a wide range of services that facilitate automation, scalability, and cost optimization. DevOps engineers can easily spin up new environments, provision resources as needed, and manage infrastructure as code, providing the flexibility required to adapt to changing business requirements. A practical-based DevOps course can make you master cloud infrastructure management.

Application Performance Optimization:

Through meticulous monitoring and analysis, DevOps identifies performance bottlenecks in applications and takes targeted optimization measures. By fine-tuning the codebase, optimizing

R.M.K GROUP OF INSTITUTIONS

database queries, and adjusting resource allocation, DevOps ensures that applications deliver a smooth and responsive user experience. The result is improved application performance, reduced latency, and increased customer satisfaction.

Streamlined Incident Management:

In a fast-paced digital landscape, incidents are bound to occur. DevOps enables rapid detection, response, and resolution of incidents through automated incident management processes. Teams receive real-time alerts, and automated remediation actions are triggered, reducing downtime and mitigating the impact on users. Incident management becomes more proactive and efficient, ensuring that services are restored promptly.

Enhanced Customer Experience:

At the heart of every DevOps effort lies the goal of delivering an exceptional customer experience. By incorporating customer feedback, deploying bug fixes promptly, and maintaining high application performance, DevOps ensures that end-users receive a product that meets their expectations. An enhanced customer experience leads to increased user satisfaction, loyalty, and positive word-of-mouth, driving business growth.

16. Assessment Schedule

Tentative schedule for the Assessment During 2025-2026 Odd Semester

S. No.	Name of the Assessment	Start Date	End Date	Portion
1	Unit Test 1			Unit 1
2	IAT 1			Unit 1 & 2
3	Unit Test 2			Unit 3
4	IAT 2			Unit 3 & 4
5	Revision 1			Unit 5, 1 & 2
6	Revision 2			Unit 3 & 4
7	Model			All 5 Units

INSTITUTIONS

17. Text Books & References

TEXTBOOKS:

1. Deepak Gaikwad, Viral Thakkar, "DevOps Tools: from Practitioner's Point of View", Wiley, 2019.
2. Jennifer Davis, Ryn Daniels, "Effective DevOps", O'Reilly Media, 2016.

REFERENCES:

1. Gene Kim, Jez Humble, Patrick Debois, "The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations", IT Revolution Press, 2016.
2. Jez Humble, Gene Kim, "Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation", Addison-Wesley, 2010.
3. Yevgeniy Brikman, "Terraform: Up & Running: Writing Infrastructure as Code", O'Reilly Media, 2019.
4. Joseph Muli, "Beginning DevOps with Docker", Packt Publishing, 2018.

18. Mini Project Suggestions

1. Build a Version Control System (VCS) ? (CO2, K3)

Using, a Version Control System helps you keep track of all the different versions, making it easy to switch between them and explore the history of your code changes. Even independent branches can be made to work on certain features without affecting the main code. At first, setting up a repository could seem difficult, but it is more important on application building.

2. Learn Source Code Management and Git Branching (CO2, K3)

In the world of DevOps, knowing how to manage source code is a must-have skill, and Git is one of the most popular version control systems out there. Git's branching functionality, which enables you to work on many aspects of your project at once, is one of its most distinctive features. To get really good at using Git's branching abilities, work on a project that lets you practice creating branches, merging them, and more.

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.